EP-392: UG Project-I

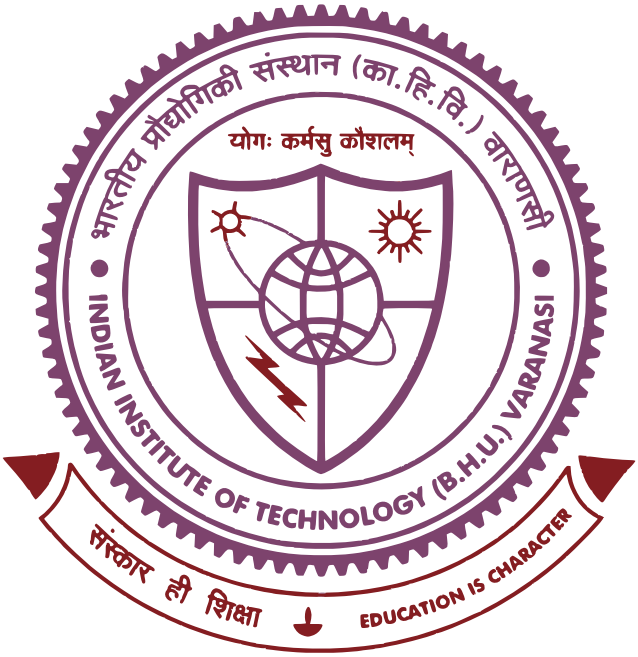
Downscaling Approach To Produce High Spatial Soil Moisture Products Using Satellite Data

Mayur Kunnitigi Matada - 23174017

Supervisor: Prof. Rajendra Prasad

Department Of Physics

April 2026



Contents

Abstract 4

Executive Summary 5

1 Project Motivation and Scope 6
- 1.1 What Soil Moisture Is . . . 6
- 1.2 Why Downscaling Is Needed for Agriculture . . . 6
- 1.3 Why Spain Was Chosen . . . 7
- 1.4 Problem Definition . . . 7
- 1.5 Scope Boundaries . . . 8

2 Data Sources and Provenance 9
- 2.1 In Situ Labels: ISMN REMEDHUS . . . 9
- 2.2 Satellite and Reanalysis Predictors . . . 10
- 2.3 Quality Constraints and Alignment Logic . . . 12

3 Study Area, Temporal Scope, and Sampling Geometry 13
- 3.1 Baseline Scope . . . 13
- 3.2 Final Scope . . . 14
- 3.3 Patch-Centric Sampling . . . 14

4 Data Exploration Protocol and Findings 15
- 4.1 EDA Objectives and Protocol . . . 15
- 4.2 Schema and Integrity Checks . . . 16
- 4.3 Target Distribution and Physical Plausibility . . . 16
- 4.4 Temporal Coverage and Seasonality . . . 17
- 4.5 Station-Level Balance and Behavior . . . 18
- 4.6 Spatial Footprint Diagnostics . . . 19
- 4.7 Outlier Diagnostics and Robustness Implications . . . 20
- 4.8 TFRecord Patch-Level EDA . . . 21
- 4.9 EDA-Driven Preprocessing Decisions . . . 22

5 Baseline Pipeline **23**
5.1 Baseline Role in the Project . . . 23
5.2 ISMN Parsing and Label Construction . . . 24
5.3 Feature Sampling from Earth Engine . . . 25
5.4 Merge and Sanity Diagnostics . . . 27
5.5 Baseline Residual XGBoost Modeling . . . 27
5.6 Baseline Outcome . . . 29

6 Final Pipeline **30**
6.1 Design Rationale . . . 30
6.1.1 Positioning Against Prior Work . . . 30
6.2 Long-Range Label Preparation . . . 31
6.3 Dynamic, Meteorological, and Static Feature Stack . . . 32
6.3.1 Derived Variables . . . 32
6.3.2 Patch Tensor Construction . . . 33
6.4 TFRecord Export and PyTorch Ingestion . . . 34
6.5 Normalization and Seasonal Channels . . . 36

7 Model Architecture: MultiModalSoilNet **37**
7.1 Overview . . . 37
7.2 Branch A: Dynamic Spatial CNN . . . 40
7.3 Branch B: Static Center MLP . . . 41
7.4 Branch C: Temporal LSTM . . . 42
7.5 Fusion and Output Head . . . 43

8 Sequence Construction and Data Contracts **45**
8.1 Tensor Contracts . . . 45
8.2 Station-Aware Chronological Sequence Builder . . . 45

9 Training Objective, Optimization, and Tuning **48**
9.1 Robust Weighted Huber Loss . . . 48
9.2 Outlier Trimming by Quantiles . . . 49
9.3 Optimization Stack . . . 49
9.4 Hyperparameter Search Program . . . 51

10 Evaluation Protocol and Scientific Rigor **52**
10.1 Primary vs Secondary Split . . . 52
10.2 Metrics . . . 52
10.3 Calibration Layer . . . 53

11 Results **55**
11.1 Baseline Results . . . 55
11.2 Final Model Results . . . 55
11.3 Interpretation . . . 56
11.4 Comparative Summary . . . 57

12 Limitations **58**
12.1 Measurement and Representation Limits . . . 58
12.2 Sampling Bias and Coverage Limits . . . 58
12.3 Temporal and Sensor Availability Constraints . . . 59
12.4 Model Complexity Tradeoff . . . 59

13 Future Work **60**
13.1 Further Downscaling from 1 km to 500 m . . . 60
13.2 Expansion to Global Generalization . . . 60
13.3 Improvement of Unseen-Station Performance . . . 61

14 Conclusion **62**

Abstract

This report tells the story of the Spain branch of the UG_SM project: what we built, what worked, what did not, and why. The goal is simple to state but hard in practice: estimate near-surface soil moisture at finer spatial detail, while still generalizing well to stations the model has never seen. To do that, the pipeline combines REMEDHUS in situ measurements with satellite and meteorological predictors from multiple sources. Throughout the report, results are tied to concrete engineering decisions so the science stays connected to implementation.

The workflow has two clear stages. We start with a residual XGBoost baseline to validate the data pipeline and set a trustworthy reference. We then move to a multimodal deep-learning system with spatial patches, short sequence memory, explicit feature routing, and a calibration layer. Under the strict unseen-station protocol, performance improves substantially over baseline and reaches the project target. Code snippets are included on purpose, so claims are easy to trace back to reproducible procedures.

Executive Summary

Spain was selected as the main test bed because REMEDHUS gives a useful balance of station density, record length, and hydro-climatic variability. In other words, it is challenging enough to be meaningful, but still practical to work with end-to-end. The project followed a two-stage strategy:

1. Build and validate a robust baseline with explicit quality checks and station-split evaluation.
2. Scale to a patch-based multimodal deep model with explicit spatial and temporal context.

Key baseline result:

- RMSE: 0.0830 m^3/m^3
- R^2: 0.3619

Key final result (selected strict protocol):

- Unseen-station R^2: 0.7016
- Unseen-station NRMSE: 6.21%

Temporal prediction result (secondary diagnostic protocol):

- Temporal-split R^2: 0.8656
- Temporal-split NRMSE: 3.53%

Compared with the baseline, the final system adds three practical capabilities: (i) local neighborhood learning through convolutional branches, (ii) short-term hydrologic memory with LSTM sequence encoding, and (iii) residual learning around a physically meaningful microwave anchor, followed by calibration. Together, these changes make the model much more useful for deployment at previously unseen sites.

Chapter 1

Project Motivation and Scope

1.1 What Soil Moisture Is

Soil moisture is the water held in soil pore space, usually reported as volumetric water content in m^3/m^3. Near the surface (roughly the top few centimeters), it changes quickly with rainfall, evaporation demand, vegetation uptake, and soil texture. Even though that layer is thin, it strongly affects land-atmosphere exchange and early plant stress.

So in practice, soil moisture is more than a hydrology variable. It is a decision signal. Farmers, irrigation managers, and drought analysts use it to answer very practical questions: when to irrigate, how much to apply, and where to intervene first.

1.2 Why Downscaling Is Needed for Agriculture

Many satellite soil moisture products are too coarse for field-scale decisions. A single coarse pixel can blend crop parcels, bare soil, roads, and varying terrain in one value. That limits direct use in precision agriculture.

Downscaling helps bridge that gap. It combines auxiliary predictors (radar, optical, thermal, meteorology, terrain, soil texture, and others) to estimate finer-resolution soil moisture patterns that are easier to act on. This direction is consistent with CNN-based SMAP downscaling and multi-sensor fusion work [1, 5]. For agriculture, the motivation is straightforward:

- **Irrigation scheduling**: identify local dry pockets instead of irrigating uniformly.
- **Input efficiency**: reduce over-application of water and pumping cost.
- **Yield protection**: react earlier to emerging moisture deficits.
- **Risk management**: improve drought and heatwave preparedness at parcel scale.

Because ground stations are sparse and costly, good generalization to unseen locations is especially important for real deployment.

1.3 Why Spain Was Chosen

Spain is a strong region for supervised geospatial soil-moisture modeling because it combines good station quality, environmental diversity, and workable data availability. The REMEDHUS network is central in this project. Stations are geographically coherent, the agricultural setting is relevant, and seasonal wetting-drying cycles are strong enough to teach the model while still exposing its weak points.

So this was not just a convenience choice. Spain gives real scientific leverage: we can test transfer to unseen stations while checking that behavior remains physically reasonable. A setting with both seasonality and land-surface heterogeneity is ideal for comparing pointwise baselines against spatial-temporal deep models.

1.4 Problem Definition

From the start, we treat SMAP as a physically meaningful first guess and train the model to predict corrections around it. We prefer this over direct absolute prediction because correction targets are usually easier to optimize and often calibrate better.

This residual framing also lines up with recent literature trends: sequence-based SMAP reconstruction emphasizes temporal continuity [2], while large-scale machine-learning products emphasize strong practical baselines over long horizons [4].

Let the true daily near-surface volumetric soil moisture at station i and day t be $y_{i,t}$. Let $a_{i,t}$ denote the center-pixel SMAP-based anchor estimate. The modeling objective is to learn a function f such that

$$\hat{y}_{i,t} = a_{i,t} + f(\mathbf{x}_{i,t}^{\text{dyn}}, \mathbf{x}_{i}^{\text{stat}}, \mathbf{s}_{i,t}), \tag{1.1}$$

where:

- $\mathbf{x}_{i,t}^{\text{dyn}}$ represents dynamic multi-sensor variables,
- $\mathbf{x}_{i}^{\text{stat}}$ represents static soil and terrain attributes,
- $\mathbf{s}_{i,t}$ represents short sequence history features.

In residual form, the target is

$$r_{i,t} = y_{i,t} - a_{i,t}, \tag{1.2}$$

and the model predicts $\hat{r}_{i,t}$, then reconstructs $\hat{y}_{i,t} = a_{i,t} + \hat{r}_{i,t}$.

1.5 Scope Boundaries

This report stays focused on the Spain pipeline. It does not include cross-country transfer experiments, global gridded deployment, or physics-constrained PDE coupling. The report covers:

- in situ acquisition and QC,
- Earth Engine feature stack design,
- baseline benchmarking,
- deep model architecture and training,
- calibration and evaluation,
- reproducibility and implementation details.

Chapter 2

Data Sources and Provenance

2.1 In Situ Labels: ISMN REMEDHUS

Ground-truth labels come from the International Soil Moisture Network package for Spain, specifically REMEDHUS stations. The pipeline parses CEOP-like station files and applies strict quality filters. Measurements are restricted to the 0–5 cm layer to match the surface products used as predictors.

The station files encode:

- station coordinates and elevation,
- depth interval,
- date-time records,
- measured value,
- ISMN quality and provider-level flags.

The project keeps only rows flagged as "good" and then aggregates timestamps to daily values.

That is a deliberate choice, not a cosmetic one. In situ labels are the supervised target, so low-quality or depth-mismatched values directly inject noise into the objective. Restricting to good 0–5 cm observations keeps label meaning aligned with both predictor channels and the anchor.

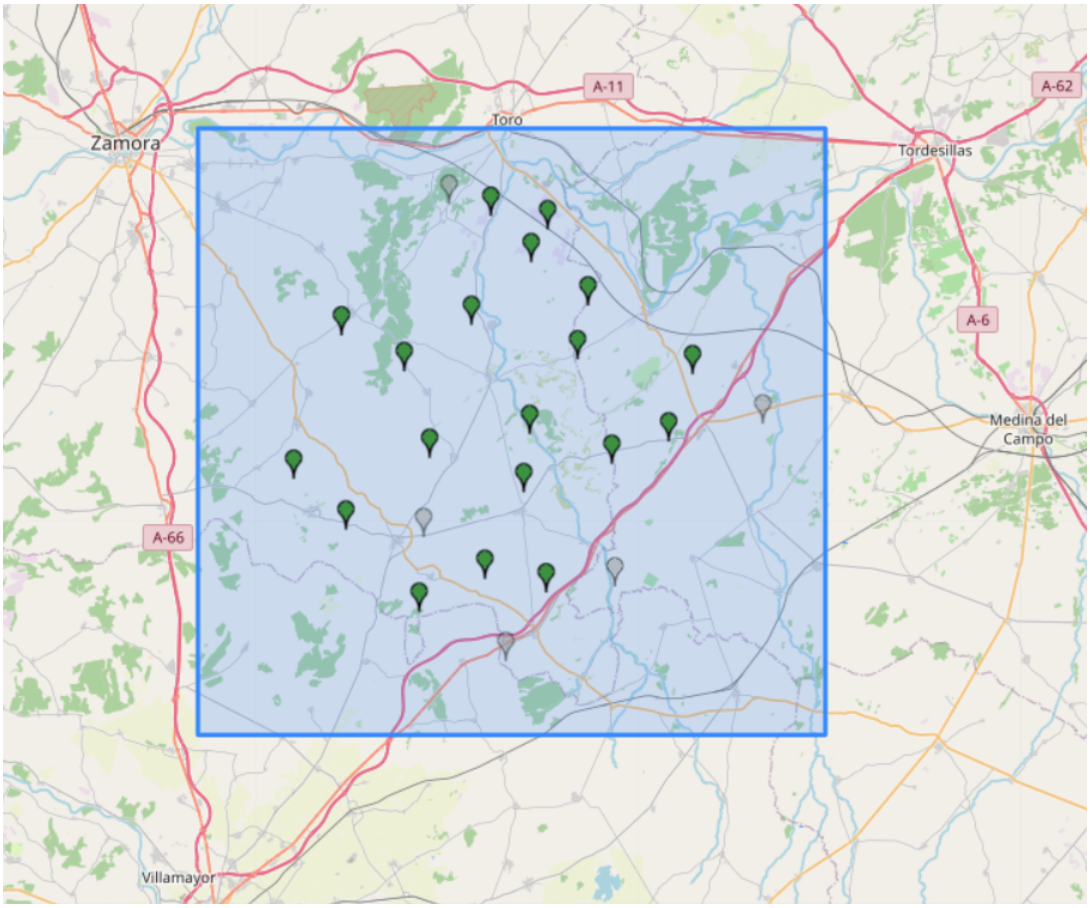


Figure 2.1: Area of interest (AOI) and REMEDHUS station distribution used in the Spain study region.

2.2 Satellite and Reanalysis Predictors

The predictor stack blends optical, radar, thermal, passive microwave, meteorological, and static terrain/soil information. The major sources are:

- Sentinel-2 SR harmonized (vegetation/water indices in baseline),
- Sentinel-1 GRD (VV, VH, ratio, incidence angle),
- MODIS LST and LAI,
- MODIS NDVI and albedo (in final stack),
- SMAP L4 soil moisture,
- ERA5-Land daily fields (precipitation, temperatures, derived variables),
- SoilGrids texture fractions (0–5 cm),

- NASADEM terrain layers.

Product documentation and methodological references include Sentinel-1/2 mission material, SMAP L4 and ERA5-Land documentation, and standard references for SoilGrids and NASADEM.

This mix is intentional: each source family captures different physics and tends to fail in different ways. No single sensor is reliable across all land and weather states, so we rely on fusion to reduce single-source failure modes. In practice, the source families contribute as follows:

- **Sentinel-2 optical indices**: provide vegetation and canopy-water context (NDVI/NDWI), which helps explain evapotranspiration demand and vegetation-mediated moisture dynamics. Optical data are informative but cloud-sensitive, so they are not used alone.
- **Sentinel-1 radar backscatter**: contributes cloud-robust structural/moisture-sensitive response (VV, VH, ratio). Radar helps when optical imagery is unavailable and captures roughness/structure interactions that optical bands miss.
- **MODIS LST and LAI/NDVI/albedo**: provides thermal stress and canopy state information at stable revisit cadence. Thermal state is strongly linked to surface drying rates, while vegetation and albedo moderate energy partitioning.
- **SMAP L4 soil moisture**: used as a physically meaningful microwave anchor. It carries direct soil-moisture signal but at coarse resolution, so the model is trained to correct local bias via residual learning.
- **ERA5-Land variables**: contribute forcing context (precipitation, temperature, dew point) and derived process variables (for example VPD/API). This helps model time-lag effects and meteorological control not always visible in instantaneous imagery.
- **SoilGrids texture fractions**: encode local storage and drainage behavior (clay/sand/silt), which control infiltration and retention differences between sites.
- **NASADEM topography**: elevation/slope/aspect provide terrain controls on runoff redistribution, radiation exposure, and local moisture persistence.

Taken together, the dynamic channels explain short-term variability, while static channels explain persistent site behavior. The architecture is designed to use both without letting either one dominate training.

2.3 Quality Constraints and Alignment Logic

Three constraints are critical:

1. Label trustworthiness: retain only high-quality in situ observations.
2. Temporal consistency: aggregate to daily support and align with daily/near-daily predictors.
3. Row-preservation robustness: prevent silent sample loss from sparse revisit and masking gaps.

The third point is especially important in geospatial ETL. Image masking can silently drop sampled rows in patterns that look random but are often seasonal or weather-dependent.

So these checks are about validity, not only cleanliness. Weak label filtering pushes the model toward noisy targets. Loose temporal alignment means predictors and labels describe different physical states. Ignoring row preservation can introduce geographically biased missingness and make metrics look better (or worse) for the wrong reasons.

Chapter 3

Study Area, Temporal Scope, and Sampling Geometry

Chapter 3 sets the spatial and temporal frame that every later result depends on. These choices are not just logistical. They affect how representative the training set is, how much seasonal signal the model sees, and how fairly station-transfer performance can be interpreted. Here, geometry and time windows were chosen to balance three practical goals: enough samples, enough environmental variability, and a workflow that remains computationally manageable.

3.1 Baseline Scope

The baseline notebook operates over a narrower period and broad AOI bounding box:

- AOI: $[-5.6, 41.0, -4.6, 41.8]$,
- start: 2019-03-01,
- end: 2020-02-29.

This baseline window is intentionally compact. It acts as a controlled proving ground for ETL validation and first-pass model behavior. A shorter period speeds up debugging and makes merge diagnostics easier to interpret because there are fewer confounding changes in sensor availability. The wider baseline AOI also helps retain enough station-day pairs after quality filters, which is important for establishing a credible reference score before moving to higher-complexity deep models.

3.2 Final Scope

The final deep-learning notebook broadens temporal coverage and uses patch geometry:

- AOI: $[-5.73, 41.09, -5.09, 41.54]$,
- start: 2015-07-01,
- end: 2025-01-01,
- CRS: EPSG:32632,
- nominal scale: 1000 m,
- patch size: 13×13.

The final setup extends across multiple years so the model sees a broader mix of wet and dry periods, seasonal transitions, and interannual variability. That matters for station generalization: a single-year window can easily overfit temporary weather regimes. The AOI is also tighter than in the baseline, so training focuses on the most relevant station support rather than unnecessary heterogeneity.

Using a fixed projected CRS (EPSG:32632) at 1000 m nominal scale keeps spatial referencing consistent across sensors and dates. That reduces geometric mismatch during sampling and helps patch tensors stay aligned for convolutional learning.

3.3 Patch-Centric Sampling

For each station-day sample, the final pipeline extracts a neighborhood patch from every channel. This gives an image-like tensor where convolutional layers can learn local gradients and mixed-pixel context. With radius 6 in each direction, the patch size is 13×13.

This patch-centric design directly addresses mixed-pixel effects. Station observations are point-like, but remote-sensing products are area averages. Looking only at the center pixel can miss important context. Neighborhood patches let the model see whether the center sits in a coherent wet patch, a transition zone, or a heterogeneous landscape.

Patch size is a tradeoff. Bigger patches can capture broader context but increase memory and may dilute local relevance. Smaller patches are cheaper but may miss useful gradients. The chosen 13×13 size is a practical middle ground for repeated tuning.

Chapter 4

Data Exploration Protocol and Findings

Chapter 4 summarizes the exploratory data analysis (EDA) run in `spain_explo.ipynb`. The goal was not just to make plots, but to verify data integrity, understand temporal and station sampling structure, characterize tail behavior, and validate the ML-ready patch dataset before training.

4.1 EDA Objectives and Protocol

The notebook follows a staged protocol for supervised geospatial time-series modeling:

1. parse CEOP-like station files with quality filtering (flag = G),
2. collapse duplicate sensors at exact timestamps and aggregate to daily labels,
3. verify schema, missingness, duplicate structure, and physical bounds,
4. analyze target distribution and temporal behavior,
5. inspect station-level sample balance and spatial footprint,
6. quantify tail behavior for robust objective design,
7. run a second EDA pass on sampled TFRecord patches (center and neighborhood statistics).

Two data representations are inspected:

- label table from raw station files: `Data/Spain/REMEDHUS/*/*.stm`,
- ML-ready patch set: `Data/Spain/ML_ready_1km_patches_spain.tfrecord.gz`.

4.2 Schema and Integrity Checks

After parsing and daily aggregation, the resulting label table has:

- 20 parsed station files,
- 72,265 station-day rows,
- 20 unique stations,
- time range from 2014-05-01 to 2025-01-01.

Core integrity diagnostics from the notebook are:

- missing values across key columns: 0,
- exact duplicate rows: 0,
- duplicates on (`point_id`, `timestamp`): 0,
- physical bound violations (`sm<0`, `sm>1`): 0.

These checks show that the supervised label stream is internally consistent before feature merging and split construction.

4.3 Target Distribution and Physical Plausibility

The notebook inspects `sm` using histogram+KDE, boxplot, ECDF, and quantile summaries. Key distribution statistics are:

- mean = 0.1217,
- median = 0.0990,
- 1st percentile = 0.0001,
- 99th percentile = 0.4089,
- min/max = 0.0000 / 0.5030.

The distribution is right-skewed with a meaningful upper tail, consistent with episodic wet conditions. That shape supports the later use of robust residual modeling.

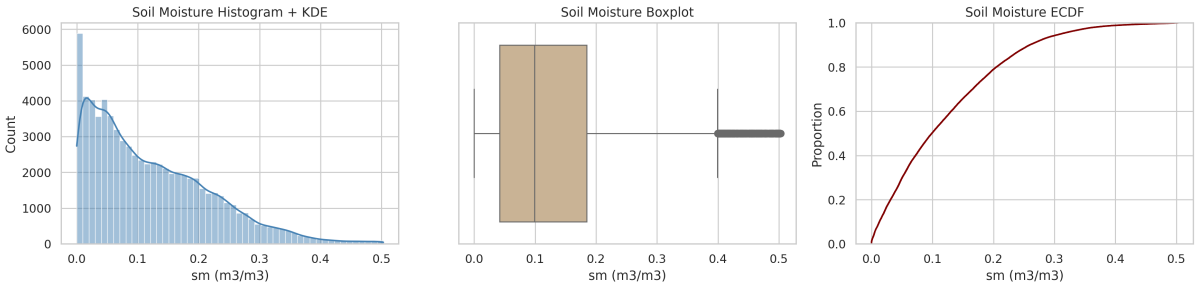


Figure 4.1: Spain EDA target distribution diagnostics from `spain_explo.ipynb`: histogram with KDE, boxplot, and ECDF for daily soil moisture labels.

4.4 Temporal Coverage and Seasonality

Temporal diagnostics in `spain_explo.ipynb` include daily count trajectories, month-wise means, year-wise means, and month-year heatmaps. Main findings are:

- daily record count summary: min = 1, median = 19, max = 20 station records/day,
- clear seasonal cycle in monthly means,
- highest monthly mean in January (0.1763),
- lowest monthly mean in August (0.0653).

Overall, these diagnostics confirm a sustained wetting-drying seasonal signal that is well suited for sequence-aware models.



Figure 4.2: Temporal coverage diagnostics from `spain_explo.ipynb`: daily record count, monthly mean soil moisture, and yearly mean soil moisture.

4.5 Station-Level Balance and Behavior

Station-level EDA covers sample density, per-station statistics, and trajectories for the densest stations. The sample count distribution is balanced enough for strict station-disjoint evaluation:

- minimum samples/station: 1,279,
- median samples/station: 3,823,
- maximum samples/station: 3,860.

Time-series plots for the densest stations show coherent behavior and no obvious station-specific artifacts that would invalidate transfer testing.

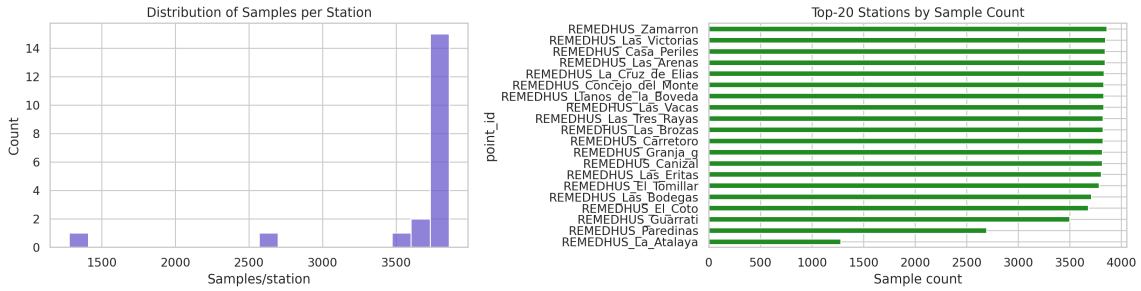


Figure 4.3: Station sample-balance diagnostics from `spain_explo.ipynb`: overall sample-count distribution and top-20 stations by sample count.

4.6 Spatial Footprint Diagnostics

Spatial diagnostics aggregate unique station coordinates and map station-level mean/sample summaries. Observed coordinate ranges are:

- latitude: 41.14894 to 41.45586,
- longitude: -5.59190 to -5.22474.

These coordinate ranges match the selected REMEDHUS AOI and support geographically coherent station-transfer experiments.

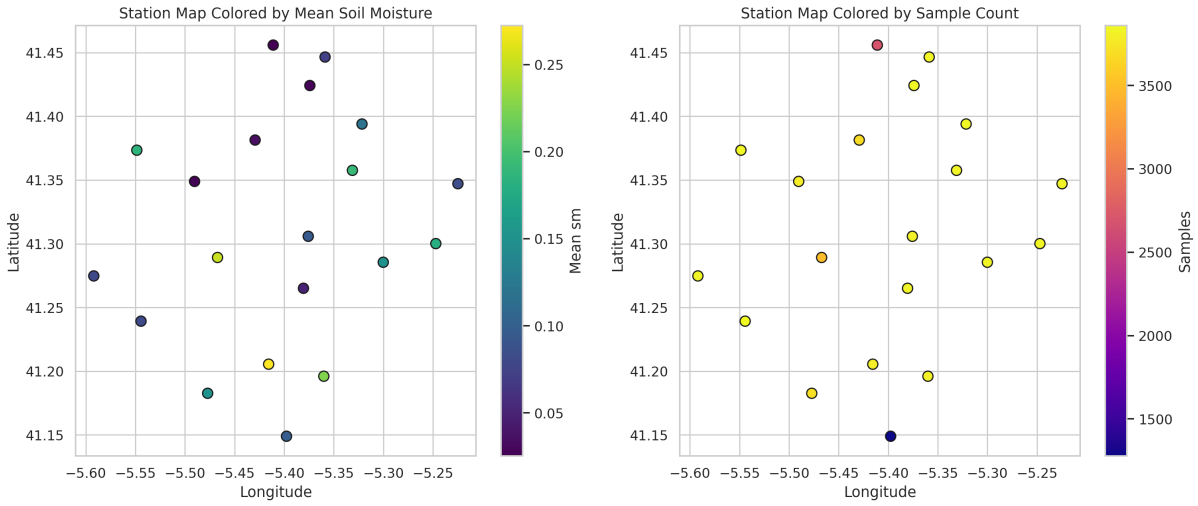


Figure 4.4: Spatial footprint diagnostics from `spain_explo.ipynb`: station map colored by mean soil moisture and by per-station sample count.

4.7 Outlier Diagnostics and Robustness Implications

Tail-risk diagnostics use the 1%/99% quantile rule:

- quantile bounds: $q_{0.01} = 0.0001$, $q_{0.99} = 0.4089$,
- potential tail points: 1,433 of 72,265 rows (1.98%).

These findings directly informed optimization choices later in the report:

- weighted robust loss,
- optional residual trimming by quantiles,
- calibration-aware evaluation.

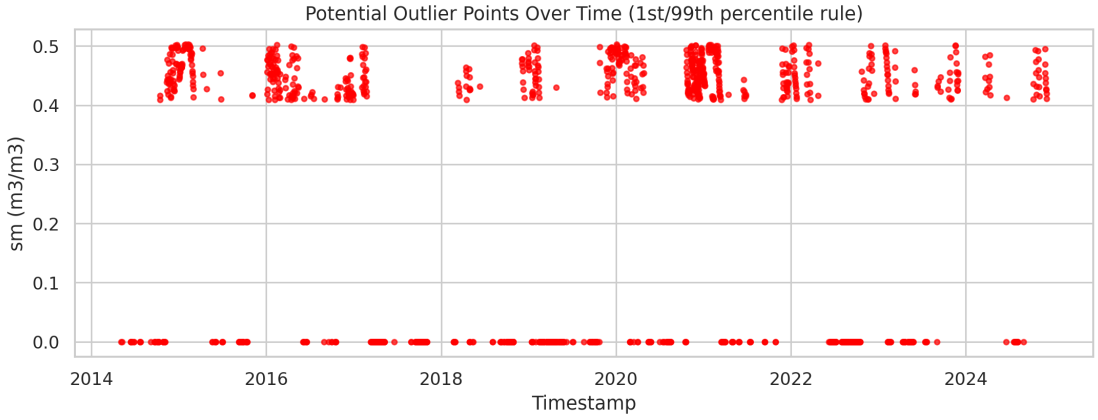


Figure 4.5: Tail-point diagnostic from `spain_explo.ipynb` using the 1%/99% quantile rule, showing potential extreme-label observations over time.

4.8 TFRecord Patch-Level EDA

The notebook includes a second EDA pass on `Data/Spain/ML_ready_1km_patches_spain.tfrecord.gz` (3,000 sampled records):

- sampled records: 3,000,
- unique stations represented: 20,
- sampled time range: 2015-07-01 to 2015-11-27,
- center-pixel missingness across listed channels: 0% in the sampled set.

Center-pixel correlation checks (SMAP, NDVI, VV, VH, LST, precip, VPD, API, Albedo) show physically plausible cross-sensor structure, including strong LST–VPD and VV–VH coupling. This is consistent with the multimodal routing strategy used in the final model.

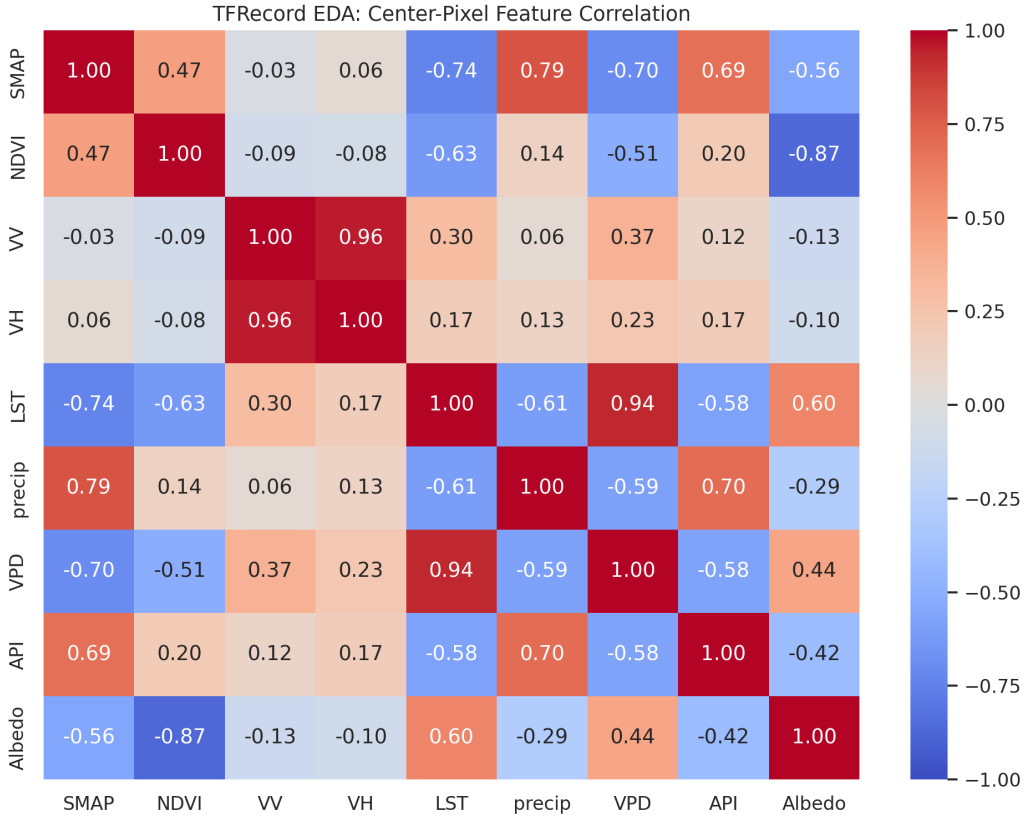


Figure 4.6: TFRecord center-pixel feature correlation heatmap from `spain_explo.ipynb` (3,000 sampled records), used to verify cross-sensor consistency prior to multimodal training.

4.9 EDA-Driven Preprocessing Decisions

Based on the above findings, the preprocessing contract used in subsequent chapters is:

1. retain physically plausible target support and valid timestamps,
2. aggregate to daily support to align with predictor cadence,
3. preserve station identity for leakage-aware splits,
4. prefer residual learning around the SMAP anchor,
5. use robust handling of heavy tails during training and evaluation.

So this EDA acts as a design gate for data contracts, objective robustness, and split rigor, not just descriptive analysis.

Chapter 5

Baseline Pipeline

5.1 Baseline Role in the Project

The baseline plays two roles: it validates the end-to-end data chain, and it sets a clear lower bound for performance. Having this reference makes it much easier to trust that later deep-model gains are real and not ETL artifacts.

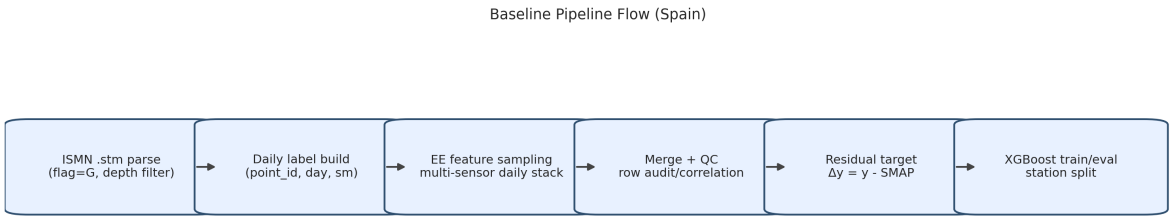


Figure 5.1: Baseline Spain pipeline flow diagram: station parsing and quality control, daily label construction, Earth Engine feature sampling, merge diagnostics, residual target definition, and station-split XGBoost training.

5.2 ISMN Parsing and Label Construction

The parser reads station metadata and measurements, applies quality filters, and aggregates to daily means. A representative implementation pattern is shown below.

```python
from pathlib import Path
import pandas as pd

def load_ismn_stm(path: Path) -> pd.DataFrame:
    lines = path.read_text(errors="ignore").splitlines()
    if len(lines) < 2:
        return pd.DataFrame()

    header = lines[0].split()
    # Expected: network, station, lat, lon, elev, depth_from,
       depth_to, ...
    station = header[1]
    lat = float(header[2])
    lon = float(header[3])
    depth_from = float(header[5])
    depth_to = float(header[6])

    if not (depth_from >= 0.0 and depth_to <= 0.05):
        return pd.DataFrame()

    rec = pd.read_csv(
        path,
        skiprows=1,
        delim_whitespace=True,
        header=None,
        usecols=[0, 1, 2, 3],
        names=["date", "time", "sm", "flag"],
    )
    rec = rec[rec["flag"] == "G"].copy()
    rec["timestamp"] = pd.to_datetime(rec["date"] + " " + rec["
       time"], errors="coerce")
    rec["sm"] = pd.to_numeric(rec["sm"], errors="coerce")
    rec = rec.dropna(subset=["timestamp", "sm"])

    rec["point_id"] = station
    rec["lat"] = lat
    rec["lon"] = lon
```

```
    return rec[["point_id", "lat", "lon", "timestamp", "sm"]]

def build_daily_labels(stm_files):
    parts = [load_ismn_stm(Path(f)) for f in stm_files]
    df = pd.concat([p for p in parts if not p.empty],
       ignore_index=True)
    df["day"] = df["timestamp"].dt.floor("D")
    return (
        df.groupby(["point_id", "lat", "lon", "day"], as_index=
           False)["sm"]
          .mean()
          .rename(columns={"day": "timestamp"})
    )
```

Listing 5.1: Representative ISMN parser for 0–5 cm good-quality rows

In the chosen baseline period, this process retains roughly nineteen valid files after depth and quality filtering.

5.3 Feature Sampling from Earth Engine

The baseline then builds a daily per-station feature table. To keep extraction robust, it uses:

- buffered windows for sparse revisit sensors,
- deterministic fallback fills for missing image windows,
- explicit unmask behavior before station sampling.

A representative Earth Engine pattern is shown below.

```python
import ee

def safe_mean(ic, start, end, bands, fallback):
    img = ic.filterDate(start, end).select(bands).mean()
    return ee.Image(ee.Algorithms.If(
        img.bandNames().size().gt(0),
        img,
        ee.Image.constant(fallback).rename(bands)
    ))

def build_feature_image(day):
    d0 = ee.Date(day)
    d1 = d0.advance(1, "day")

    s2 = safe_mean(s2_ic, d0.advance(-5, "day"), d1, ["B8", "B4",
        "B11"], [0, 0, 0])
    ndvi = s2.normalizedDifference(["B8", "B4"]).rename("NDVI")
    ndwi = s2.normalizedDifference(["B8", "B11"]).rename("NDWI")

    s1 = safe_mean(s1_ic, d0.advance(-6, "day"), d1, ["VV", "VH"
       ], [0, 0])
    vv = s1.select("VV")
    vh = s1.select("VH")
    ratio = vv.divide(vh.abs().add(1e-6)).rename("VV_VH_Ratio")

    smap = safe_mean(smap_ic, d0, d1, ["sm_surface"], [0]).rename
       ("SMAP")
    era = safe_mean(era5_ic, d0, d1, ["tp", "t2m", "d2m"], [0, 0,
        0])

    return (ndvi.addBands([ndwi, vv, vh, ratio, smap, era])
                .unmask(0)
                .set("day", d0.format("YYYY-MM-dd")))
```

Listing 5.2: Representative Earth Engine feature image builder pattern

5.4 Merge and Sanity Diagnostics

After export, features are merged with labels by station ID and timestamp. Merge quality is high (99.7%), which suggests coherent station/day alignment. Diagnostics then check:

- gap-coded zero prevalence,
- feature-target correlation structure,
- station-level time series realism,
- static feature variation across sites.

5.5 Baseline Residual XGBoost Modeling

XGBoost is used as the first supervised benchmark for practical reasons. It handles heterogeneous tabular inputs well, captures nonlinear interactions without heavy preprocessing, tolerates moderate missingness artifacts, and supports fast iteration for feature sanity checks. That makes it a good baseline before committing to a more expensive deep architecture, and it aligns with recent large-scale soil-moisture products built on strong tabular learners [4].

The baseline uses a residual target around SMAP:

$$\Delta y = y - a, \quad \hat{y} = a + \widehat{\Delta y}. \tag{5.1}$$

The residual setup is used here for the same reason as in the final model: learning corrections around a physically informed anchor usually converges faster and reduces bias drift versus predicting absolute moisture from scratch.

Selected engineered terms include rolling precipitation and thermal deltas. A representative training pattern is:

```python
import numpy as np
from xgboost import XGBRegressor
from sklearn.metrics import mean_squared_error, r2_score

train_df = df[df.point_id.isin(train_ids)].copy()
test_df  = df[df.point_id.isin(test_ids)].copy()

for part in (train_df, test_df):
    part["sm_delta"] = part["sm"] - part["SMAP"]
    part["VV_VH_Ratio"] = part["VV"] / (np.abs(part["VH"]) + 1e
       -6)

features = [
    "SMAP", "VV", "VH", "VV_VH_Ratio", "NDVI", "NDWI",
    "tp", "t2m", "d2m", "sand", "clay", "silt", "elevation"
]

model = XGBRegressor(
    n_estimators=1200,
    learning_rate=0.03,
    max_depth=6,
    subsample=0.9,
    colsample_bytree=0.9,
    reg_alpha=0.2,
    reg_lambda=2.0,
    objective="reg:squarederror",
    random_state=42,
)

model.fit(
    train_df[features],
    train_df["sm_delta"],
    eval_set=[(test_df[features], test_df["sm_delta"])],
    verbose=False,
)

pred_delta = model.predict(test_df[features])
pred = pred_delta + test_df["SMAP"].values

rmse = np.sqrt(mean_squared_error(test_df["sm"], pred))
r2 = r2_score(test_df["sm"], pred)
```

```
41 print({"RMSE": rmse, "R2": r2})
```

Listing 5.3: Representative baseline residual XGBoost training

5.6 Baseline Outcome

The benchmark result from the baseline notebook is:

- RMSE: 0.0830 m^3/m^3
- R^2: 0.3619

This is a meaningful, but still limited, benchmark and it justifies moving to a richer model family.

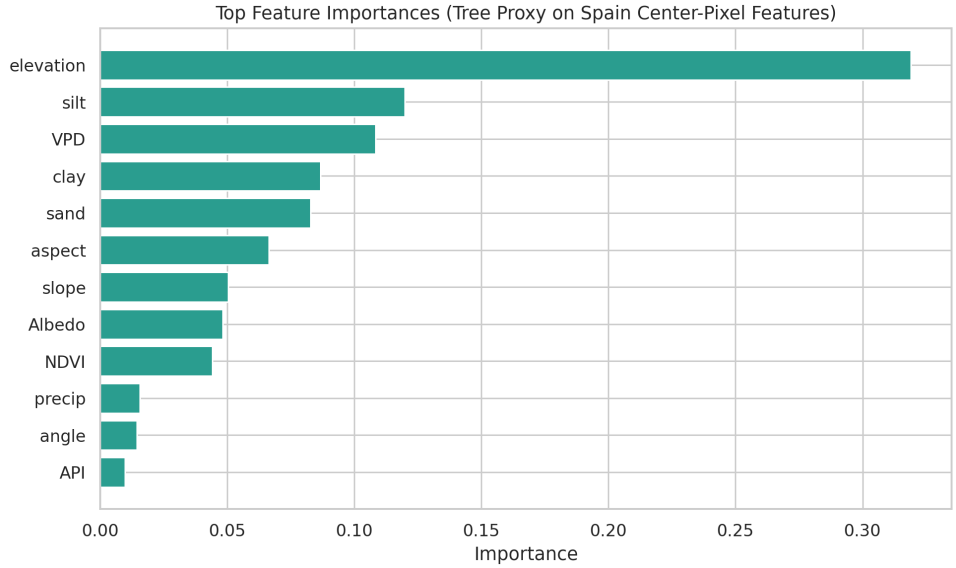


Figure 5.2: Feature-importance view from a tree-based proxy model trained on Spain center-pixel channels from sampled TFRecord records. This provides an implementation-level indicator of dominant predictors consistent with the baseline feature family.

Chapter 6

Final Pipeline

6.1 Design Rationale

The final pipeline upgrades three dimensions simultaneously:

1. **Spatial context**: learn from neighborhood patches rather than single-point rows.
2. **Temporal memory**: encode short-term forcing history via sequences.
3. **Feature specialization**: route dynamic/static/temporal signals through dedicated branches.

These choices directly target baseline limitations. Pointwise tabular models cannot explicitly capture neighborhood heterogeneity, have limited memory of delayed hydrologic response unless lag engineering is extensive, and can blend static and dynamic effects in ways that hurt transfer. The multimodal design is a deliberate response to those issues, combining strengths usually treated separately in CNN-focused downscaling and LSTM-focused sequence modeling studies [1, 2, 5].

6.1.1 Positioning Against Prior Work

Much of the soil-moisture downscaling literature emphasizes one modeling axis at a time. CNN-oriented studies usually focus on spatial refinement from multisensor imagery and patch context, with less explicit temporal modeling in the core predictor [1, 5]. LSTM-oriented studies focus on temporal continuity and hindcasting behavior [2]. In parallel, physics-driven retrieval/inversion remains important for interpretability and sensor-consistent constraints [3], while tree-based methods like XGBoost continue to be strong tabular baselines from regional to global scale [4].

Across the references reviewed for this project, these threads are often developed separately. The model here tries to bring them together in one end-to-end residual framework: a dynamic CNN branch, a temporal LSTM branch, and static-feature fusion

around a physically meaningful microwave anchor. The intended benefit is better transfer to unseen stations by learning both *where* moisture patterns appear (spatial context) and *when* they evolve (temporal context).

6.2 Long-Range Label Preparation

The final setup keeps strict quality filtering, and adds robust unit handling plus duplicate-sensor collapse before daily aggregation. This protects label quality over the longer time horizon.

```python
def prepare_labels(raw_df):
    df = raw_df.copy()
    df = df[df["flag"] == "G"].copy()
    df["timestamp"] = pd.to_datetime(df["timestamp"], errors="coerce")
    df["sm"] = pd.to_numeric(df["sm"], errors="coerce")
    df = df.dropna(subset=["timestamp", "sm"])

    # Optional unit safeguard for percent-like values.
    if df["sm"].mean() > 1.0:
        df["sm"] = df["sm"] / 100.0

    # Collapse duplicate sensors at exact timestamp.
    df = (
        df.groupby(["point_id", "lat", "lon", "timestamp"], as_index=False)["sm"]
          .mean()
    )

    df["day"] = df["timestamp"].dt.floor("D")
    daily = (
        df.groupby(["point_id", "lat", "lon", "day"], as_index=False)["sm"]
          .mean()
          .rename(columns={"day": "timestamp"})
    )
    return daily
```

Listing 6.1: Representative robust label preparation for long horizon

6.3 Dynamic, Meteorological, and Static Feature Stack

The final Earth Engine stack includes daily or windowed composites from multiple sensors, along with derived fields such as VPD and API. This explicit fusion design is also motivated by prior multi-source deep retrieval work [1].

6.3.1 Derived Variables

Vapor pressure deficit (VPD):

$$\mathrm{VPD} = e_s(T) - e_a(T_d), \tag{6.1}$$

where T is air temperature and T_d dewpoint. A common approximation is:

$$e_s(T) = 0.6108 \exp\left(\frac{17.27T}{T + 237.3}\right). \tag{6.2}$$

Antecedent precipitation index (API):

$$\mathrm{API}_t = P_t + \lambda \cdot \mathrm{API}_{t-1}, \quad 0 < \lambda < 1. \tag{6.3}$$

The implementation sketch below shows one robust Earth Engine pattern for constructing these derived variables from daily ERA5 fields.

```python
import ee

def saturation_vapor_pressure_celsius(tc):
    return ee.Image(0.6108).multiply(
        (ee.Image(17.27).multiply(tc).divide(tc.add(237.3))).exp()
    )

def make_vpd_from_era5(era_img):
    t2m_c = era_img.select("t2m").subtract(273.15)
    d2m_c = era_img.select("d2m").subtract(273.15)
    es = saturation_vapor_pressure_celsius(t2m_c)
    ea = saturation_vapor_pressure_celsius(d2m_c)
    return es.subtract(ea).max(0).rename("VPD")

def make_api(precip_series, decay=0.85):
    api = ee.Image.constant(0)
    out = []
    for img in precip_series:
        api = ee.Image(img).add(api.multiply(decay)).rename("API")
        out.append(api)
    return out
```

Listing 6.2: Representative Earth Engine helpers for VPD and API

6.3.2 Patch Tensor Construction

For each daily image, each station point is transformed into local arrays with neighborhood operators. The resulting tensors satisfy:

$$\mathbf{x}_{i,t} \in \mathbb{R}^{C\times 13\times 13}. \tag{6.4}$$

```python
def to_patch_array(image, radius=6):
    kernel = ee.Kernel.square(radius=radius, units="pixels",
       normalize=False)
    return image.neighborhoodToArray(kernel)

def sample_station_patches(image, station_fc, scale=1000):
    arr = to_patch_array(image, radius=6)
    sampled = arr.sampleRegions(
        collection=station_fc,
        properties=["point_id", "timestamp"],
        scale=scale,
        geometries=False,
    )
    return sampled
```

Listing 6.3: Representative neighborhood-to-array sampling logic

6.4 TFRecord Export and PyTorch Ingestion

Patch samples are exported as TFRecord and read with a custom iterable dataset. The ingestion layer validates shape consistency and emits tensors used for downstream splits and sequence construction.

```python
import torch
from torch.utils.data import IterableDataset

class PatchTFRecordDataset(IterableDataset):
    def __init__(self, tfrecord_paths, channel_count, patch_size=13):
        self.paths = tfrecord_paths
        self.channel_count = channel_count
        self.patch_size = patch_size

    def parse_record(self, rec):
        x = rec["x"]  # expected flattened float array
        pid = rec["point_id"]
        ts = rec["timestamp"]
        x = torch.tensor(x, dtype=torch.float32)
        x = x.view(self.channel_count, self.patch_size, self.patch_size)
        return x, pid, ts

    def __iter__(self):
        for path in self.paths:
            for rec in iterate_tfrecord(path):
                try:
                    yield self.parse_record(rec)
                except Exception:
                    continue
```

Listing 6.4: Representative TFRecord iterable dataset sketch

6.5 Normalization and Seasonal Channels

Seasonal channels are appended:

$$\text{Day_Sin} = \sin\left(2\pi\frac{d}{365}\right), \quad \text{Day_Cos} = \cos\left(2\pi\frac{d}{365}\right), \tag{6.5}$$

where d is day-of-year.

Normalization is computed from training stations only:

$$\tilde{x}_c = \frac{x_c - \mu_c^{\text{train}}}{\sigma_c^{\text{train}} + \epsilon}. \tag{6.6}$$

This avoids station leakage.

Chapter 7

Model Architecture: MultiModalSoilNet

7.1 Overview

The final architecture is a multi-branch residual predictor. Inputs are routed by meaning, not blended blindly:

- dynamic channels: CNN branch,
- static channels: center-pixel MLP branch,
- temporal sequence channels: LSTM branch.

Fusion concatenates branch embeddings and center features, then predicts residual correction.

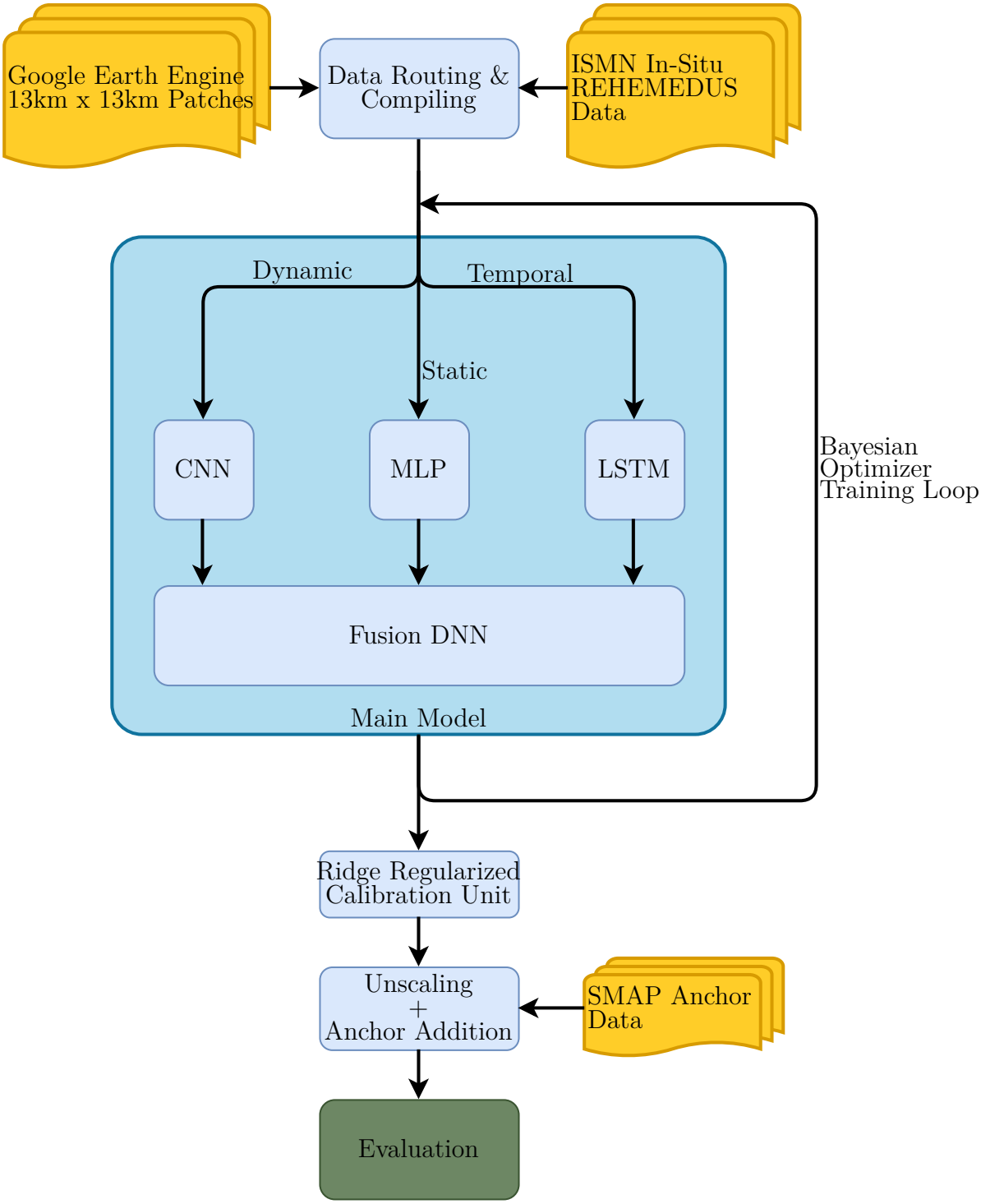


Figure 7.1: Overall MultiModalSoilNet architecture showing branch routing, fusion, and residual output around the SMAP anchor.

Routing is intentional for identifiability and optimization stability. Dynamic channels form spatial patterns in convolutions, static channels are compressed separately so they do not dominate conv activations, and temporal channels are handled as sequences rather than flattened one-day covariates.

The routing contract is explicit in code so tensor assembly stays consistent across training and inference.

```
routing = {
    "dynamic": [
        "NDVI", "VV", "VH", "VV_VH_Ratio", "LST", "SMAP",
        "Albedo", "precip", "VPD", "API", "Day_Sin", "Day_Cos"
    ],
    "static": ["clay", "sand", "silt", "elevation", "slope", "
       aspect"],
    "temporal": ["SMAP", "precip", "VPD", "API"],
}

def channel_indices(all_channels, names):
    idx = []
    for n in names:
        idx.append(all_channels.index(n))
    return idx

dyn_idx = channel_indices(all_channels, routing["dynamic"])
stat_idx = channel_indices(all_channels, routing["static"])
tmp_idx = channel_indices(all_channels, routing["temporal"])
```

Listing 7.1: Representative channel routing configuration

This design follows the physical structure of the problem. Soil moisture at a station-day is driven by multiple signal families at once: local spatial context (nearby land-cover and moisture gradients), persistent site properties (soil texture and terrain), and delayed forcing history (recent rainfall and atmospheric demand). A monolithic encoder can learn this in theory, but in practice it often overweights whichever feature family is easiest to fit early in training. Branch-based routing helps reduce that failure mode.

For deployment, the core goal is strict station transfer. That means learning mechanisms, not station-specific shortcuts. So the architecture separates: (i) what changes quickly across days and neighboring pixels, (ii) what is nearly fixed for a location, and (iii) what depends on recent history. In practice, this separation improves interpretability and usually improves convergence on heterogeneous geospatial inputs.

7.2 Branch A: Dynamic Spatial CNN

The CNN branch learns local motifs from 13×13 dynamic patches. Adaptive pooling compresses learned activation maps to a compact embedding.

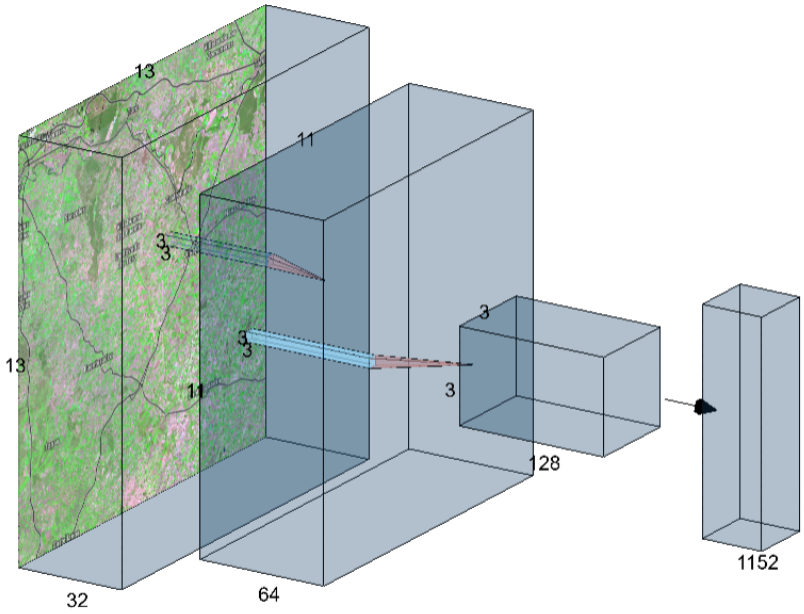


Figure 7.2: Dynamic CNN branch used to encode neighborhood patch context before fusion with static and temporal embeddings.

A convolutional neural network is used because the dynamic predictors are spatial fields, not independent scalar columns. This follows the same intuition as CNN-based SMAP downscaling work, where convolution helps recover sub-pixel structure from neighborhood context [5]. Shared kernels across the patch offer two practical benefits here:

- **Local pattern detection**: kernels learn motifs such as wet-dry boundaries, thermal-radar contrast patterns, and neighborhood gradients that are not visible from center-pixel values alone.
- **Parameter efficiency and transfer**: weight sharing means the same detector is reused across all patch locations, reducing parameter count and improving generalization to unseen stations.

For the Spain AOI, this matters because crop parcels, bare soil, and vegetation can change over short distances. A station can sit near a boundary where the center pixel alone is ambiguous. Looking at the neighborhood lets the CNN infer whether the center belongs to a coherent wet patch, a drying front, or a mixed transition.

The selected 13×13 neighborhood with stacked 3x3 convolutions gives a moderate receptive field while keeping memory cost manageable. Batch normalization and dropout

help stabilize optimization under multi-sensor variance and reduce overfitting to site-specific texture artifacts.

```python
import torch.nn as nn

class DynamicCNN(nn.Module):
    def __init__(self, in_ch, p_drop=0.15):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_ch, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.Dropout2d(p_drop),
            nn.Conv2d(64, 64, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.AdaptiveAvgPool2d((1, 1)),
        )

    def forward(self, x):
        z = self.net(x)
        return z.flatten(1)
```

Listing 7.2: Representative dynamic CNN branch

7.3 Branch B: Static Center MLP

Static channels are sampled at the center pixel and passed through a compact fully connected encoder. This allows soil texture and terrain effects to influence prediction without dominating convolutional dynamics.

The static branch uses an MLP because static predictors here are low-dimensional tabular descriptors (for example clay/sand/silt fractions and terrain attributes). These variables do not need repeated spatial convolution. What they need is a nonlinear mapping from site descriptors to hydrologic response tendency.

In practical terms, the MLP learns interactions such as:

- high clay content with low slope tending toward slower drainage,
- sand-rich profiles tending toward faster infiltration and faster drying,
- terrain-mediated differences in runoff redistribution and persistence.

Using a dedicated MLP path also avoids a common failure mode. If static channels are injected directly as constant CNN planes, they can dominate early gradients while contributing little spatial information. The center-MLP formulation keeps static effects important but controlled, which helps unseen-station transfer.

```python
class StaticMLP(nn.Module):
    def __init__(self, in_dim, hid=32, p_drop=0.1):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, hid),
            nn.ReLU(inplace=True),
            nn.Dropout(p_drop),
            nn.Linear(hid, hid),
            nn.ReLU(inplace=True),
        )

    def forward(self, x_center_static):
        return self.net(x_center_static)
```

Listing 7.3: Representative static center branch

7.4 Branch C: Temporal LSTM

The LSTM branch encodes short sequence histories of center-pixel hydro-meteorological variables.

An LSTM is used because soil moisture has memory. The response on day t depends not only on same-day forcing, but also on recent wetting and drying history. Infiltration, evapotranspiration demand, and drainage are lagged and nonlinear, so a sequence model is more suitable than a purely instantaneous regressor. This is consistent with prior SMAP prolongation work using LSTM temporal learning [2].

Why LSTM in this specific setting:

- **Lag sensitivity**: it can learn variable decay and carry-over patterns from recent precipitation and atmospheric demand.
- **Compactness**: with moderate sequence lengths and dataset size, LSTM is compute-efficient and stable.
- **Robust training**: gated recurrence helps preserve useful history while filtering irrelevant fluctuations.

Compared with hand-crafted lag features alone, the LSTM can adaptively weight which recent days matter most for each sample. Compared with heavier sequence architectures, it provides a strong accuracy-cost tradeoff for repeated station-split tuning.

```python
class TemporalEncoder(nn.Module):
    def __init__(self, in_dim, hid=48, num_layers=2, p_drop=0.15):
        super().__init__()
        self.lstm = nn.LSTM(
            input_size=in_dim,
            hidden_size=hid,
            num_layers=num_layers,
            batch_first=True,
            dropout=p_drop,
        )

    def forward(self, seq):
        _, (h_n, _) = self.lstm(seq)
        return h_n[-1]
```

Listing 7.4: Representative temporal LSTM branch

7.5 Fusion and Output Head

The final head concatenates embeddings and center vectors, then outputs one scalar residual.

Fusion is where the three information streams are reconciled into one correction decision. Conceptually, the model computes:

$$\hat{r}_{i,t} = g\left([z_{\text{cnn}},\ z_{\text{mlp}},\ z_{\text{lstm}},\ x^{\text{dyn}}_{\text{center}},\ x^{\text{stat}}_{\text{center}}]\right), \tag{7.1}$$

where each embedding contributes complementary evidence.

This matters in real edge cases. A CNN-only system may detect a wet neighborhood but miss that sandy texture implies faster drainage. An MLP-only system may encode site texture well but miss an ongoing regional drying front. An LSTM-only system may track recent rain history but miss strong same-day spatial heterogeneity. Fusion reduces these single-branch blind spots and improves robustness under strict unseen-station testing. It also acts as a practical middle path between purely physical inversion and purely tabular machine-learning baselines [3, 4].

The output head is intentionally small to limit overfitting after concatenation. Because the target is a residual correction around a physically meaningful anchor, the head only needs to learn structured adjustments rather than full moisture dynamics from scratch.

```
class MultiModalSoilNet(nn.Module):
    def __init__(self, n_dyn, n_stat, n_seq):
        super().__init__()
        self.cnn = DynamicCNN(n_dyn)
        self.stat = StaticMLP(n_stat)
        self.temporal = TemporalEncoder(n_seq)

        fusion_in = 64 + n_dyn + n_stat + 32 + 48
        self.head = nn.Sequential(
            nn.Linear(fusion_in, 96),
            nn.ReLU(inplace=True),
            nn.Dropout(0.2),
            nn.Linear(96, 48),
            nn.ReLU(inplace=True),
            nn.Linear(48, 1),
        )

    def forward(self, x_dyn_patch, x_dyn_center, x_stat_center,
       seq):
        z_sp = self.cnn(x_dyn_patch)
        z_st = self.stat(x_stat_center)
        z_tm = self.temporal(seq)
        z = torch.cat([z_sp, x_dyn_center, x_stat_center, z_st,
           z_tm], dim=1)
        return self.head(z)
```

Listing 7.5: Representative fusion model

Chapter 8

Sequence Construction and Data Contracts

8.1 Tensor Contracts

Training inputs follow strict shape definitions:

- spatial tensor: $\mathbf{x} \in \mathbb{R}^{B \times C \times 13 \times 13}$,
- temporal tensor: $\mathbf{s} \in \mathbb{R}^{B \times L \times F}$,
- residual target: $\mathbf{r} \in \mathbb{R}^{B \times 1}$.

Keeping these contracts explicit helps prevent silent bugs in multi-branch pipelines.

8.2 Station-Aware Chronological Sequence Builder

Sequences are built per station in time order, with left-padding when sequence history is short.

```python
from collections import defaultdict, deque
import numpy as np

def build_sequences(samples, seq_len, feat_cols):
    # samples expected sorted by (point_id, timestamp)
    buffers = defaultdict(lambda: deque(maxlen=seq_len))
    seq_out = []

    zero = np.zeros(len(feat_cols), dtype=np.float32)

    for row in samples:
        pid = row["point_id"]
        v = np.asarray([row[c] for c in feat_cols], dtype=np.
           float32)
        buffers[pid].append(v)

        hist = list(buffers[pid])
        if len(hist) < seq_len:
            hist = [zero] * (seq_len - len(hist)) + hist

        seq_out.append(np.stack(hist, axis=0))

    return np.stack(seq_out, axis=0)
```

Listing 8.1: Representative sequence builder

This logic preserves causality and avoids inter-station leakage.

After sequence construction, supervised tensors are assembled with optional residual trimming and robust sample weighting.

```python
import numpy as np

def make_supervised_chunk(rows, q_low=None, q_high=None):
    y = np.asarray([r["sm"] for r in rows], dtype=np.float32)
    a = np.asarray([r["SMAP_center"] for r in rows], dtype=np.
       float32)
    residual = y - a

    keep = np.ones_like(residual, dtype=bool)
    if q_low is not None and q_high is not None:
        lo = np.quantile(residual, q_low)
        hi = np.quantile(residual, q_high)
        keep = (residual >= lo) & (residual <= hi)

    w = 1.0 / (np.abs(residual) + 0.02)
    w = w / (w.mean() + 1e-8)

    return {
        "x": np.asarray([r["x"] for r in rows], dtype=np.float32)
           [keep],
        "seq": np.asarray([r["seq"] for r in rows], dtype=np.
           float32)[keep],
        "y_res": residual[keep],
        "anchor": a[keep],
        "w": w[keep],
    }
```

Listing 8.2: Representative supervised chunk construction

Chapter 9

Training Objective, Optimization, and Tuning

9.1 Robust Weighted Huber Loss

Pure MSE was not used as the main objective because it can be too sensitive to occasional sensor noise and geolocation mismatch outliers. Pure MAE is robust, but often less smooth to optimize near convergence. Huber gives a practical compromise: quadratic near zero error, linear for large errors.

For prediction error $e = \hat{r} - r$ and threshold δ, Huber loss is:

$$\ell_\delta(e) = \begin{cases} \frac{1}{2}e^2, & |e| \le \delta \\ \delta\left(|e| - \frac{1}{2}\delta\right), & |e| > \delta \end{cases} \tag{9.1}$$

Weighted objective:

$$\mathcal{L} = \frac{1}{N}\sum_{n=1}^{N} w_n \ell_\delta(e_n), \tag{9.2}$$

where w_n downweights extreme residual examples.

Weighting is included so a small number of extreme residuals do not dominate gradient updates and hurt average-case station transfer.

9.2 Outlier Trimming by Quantiles

Residual targets can be trimmed using training quantiles:

$$r \in [q_{\text{low}}, q_{\text{high}}]. \tag{9.3}$$

Trimming is configurable and tuned. Too aggressive and real extremes are removed; too weak and the model can overfit noise spikes.

9.3 Optimization Stack

AdamW is chosen deliberately over plain SGD and classic Adam. Compared with SGD, AdamW is usually more stable under heterogeneous feature scales and multimodal fusion. Compared with classic Adam, decoupled weight decay gives cleaner regularization and lowers the tendency to overfit branch-specific parameters.

Key mechanics:

- AdamW optimizer,
- gradient clipping,
- optional automatic mixed precision,
- early stopping on validation R^2,
- batch-level weighting and robust loss.

Why each component is included:

- **AdamW**: stable convergence for mixed CNN/MLP/LSTM parameter groups.
- **Gradient clipping**: prevents occasional exploding updates in recurrent or deep fusion paths.
- **AMP**: speeds training and lowers memory usage on CUDA while preserving sufficient numeric fidelity.
- **Early stopping**: selects the best generalization point before overfit to station-specific idiosyncrasies.

```python
import torch
from torch.cuda.amp import autocast, GradScaler

def train_loop(model, loader, optimizer, delta=0.05, use_amp=True):
    model.train()
    scaler = GradScaler(enabled=use_amp)
    meter = 0.0

    for batch in loader:
        x_patch, x_dyn_c, x_stat_c, seq, y_res, w = batch
        optimizer.zero_grad(set_to_none=True)

        with autocast(enabled=use_amp):
            pred = model(x_patch, x_dyn_c, x_stat_c, seq).squeeze(1)
            err = pred - y_res
            abs_err = err.abs()
            huber = torch.where(
                abs_err <= delta,
                0.5 * err * err,
                delta * (abs_err - 0.5 * delta),
            )
            loss = (w * huber).mean()

        scaler.scale(loss).backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        scaler.step(optimizer)
        scaler.update()

        meter += float(loss.detach().cpu())

    return meter / max(len(loader), 1)
```

Listing 9.1: Representative training loop skeleton

9.4 Hyperparameter Search Program

Tuning follows a staged sequence rather than one-pass random search:

1. baseline configuration,
2. Optuna station-split trials,
3. rerun with best trial,
4. polishing run,
5. aggressive refinement profile,
6. temporal fallback only if target unmet.

This staged strategy improves selection stability and helps avoid over-committing to fragile outlier trials.

Chapter 10

Evaluation Protocol and Scientific Rigor

10.1 Primary vs Secondary Split

Two split families are reported:

- station split (primary): test stations unseen in training,
- temporal split (secondary): future times on known stations.

Station split is stricter and better aligned with deployment transfer.

That ordering is intentional. The headline metric is station split because practical deployment often targets places without local in situ training labels. Temporal split is kept as a secondary diagnostic for easier known-station conditions.

10.2 Metrics

Reported metrics include:

- coefficient of determination R^2,
- RMSE,
- normalized RMSE (NRMSE).

Definitions:

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{n=1}^{N}(y_n - \hat{y}_n)^2}, \tag{10.1}$$

$$R^2 = 1 - \frac{\sum_n (y_n - \hat{y}_n)^2}{\sum_n (y_n - \bar{y})^2}, \tag{10.2}$$

$$\mathrm{NRMSE} = \frac{\mathrm{RMSE}}{\max(y) - \min(y)}. \tag{10.3}$$

The utility below computes evaluation metrics in a consistent, reproducible way across validation and test partitions.

```python
import numpy as np
from sklearn.metrics import r2_score, mean_squared_error

def eval_metrics(y_true, y_pred):
    rmse = float(np.sqrt(mean_squared_error(y_true, y_pred)))
    r2 = float(r2_score(y_true, y_pred))
    nrmse = float(rmse / (np.max(y_true) - np.min(y_true) + 1e-8)
        )
    return {"RMSE": rmse, "R2": r2, "NRMSE": nrmse}
```

Listing 10.1: Representative evaluation utility

10.3 Calibration Layer

After raw residual reconstruction, a quadratic calibrator is used to adjust systematic bias:

$$\hat{y} = c_0 + c_1 a + c_2 r + c_3 ar + c_4 a^2 + c_5 r^2, \tag{10.4}$$

where a is anchor and r is predicted residual. Coefficients are fit with ridge regularization.

```python
import numpy as np
from sklearn.linear_model import Ridge

def poly2_features(anchor, resid):
    return np.column_stack([
        np.ones_like(anchor),
        anchor,
        resid,
        anchor * resid,
        anchor * anchor,
        resid * resid,
    ])

def fit_calibrator(a_train, r_train, y_train, alpha=1e-2):
    X = poly2_features(a_train, r_train)
    reg = Ridge(alpha=alpha, fit_intercept=False)
    reg.fit(X, y_train)
    return reg

def apply_calibrator(cal, a, r):
    X = poly2_features(a, r)
    return cal.predict(X)
```

Listing 10.2: Representative quadratic calibration fit/apply

Chapter 11

Results

11.1 Baseline Results

Baseline performance from the Spain baseline notebook:

- RMSE: 0.0830 m^3/m^3
- R^2: 0.3619

11.2 Final Model Results

Final selected strict split result:

- split type: station,
- best validation R^2: 0.7016,
- anchor R^2: 0.1873,
- NRMSE: 6.21%,
- selected best epoch: 10.

Additional diagnostic result:

- temporal split R^2: 0.8656,
- temporal split NRMSE: 3.53%.

11.3 Interpretation

The large jump from baseline to final strict station split supports the central claim of this report: patch-level spatial context, sequence memory, and feature routing meaningfully improve transfer performance. The higher temporal-split score is expected, since station identity and static context are less novel there.

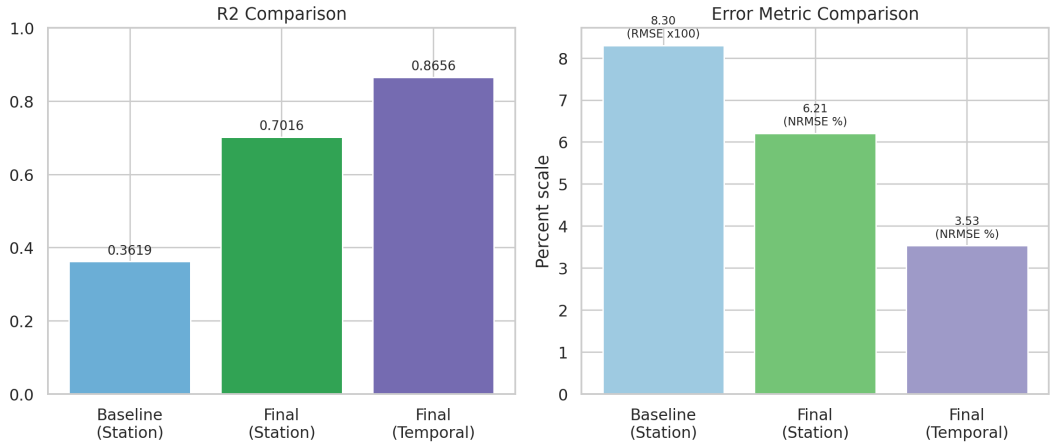


Figure 11.1: Comparison chart of baseline and final model outcomes using report metrics (station and temporal protocols).

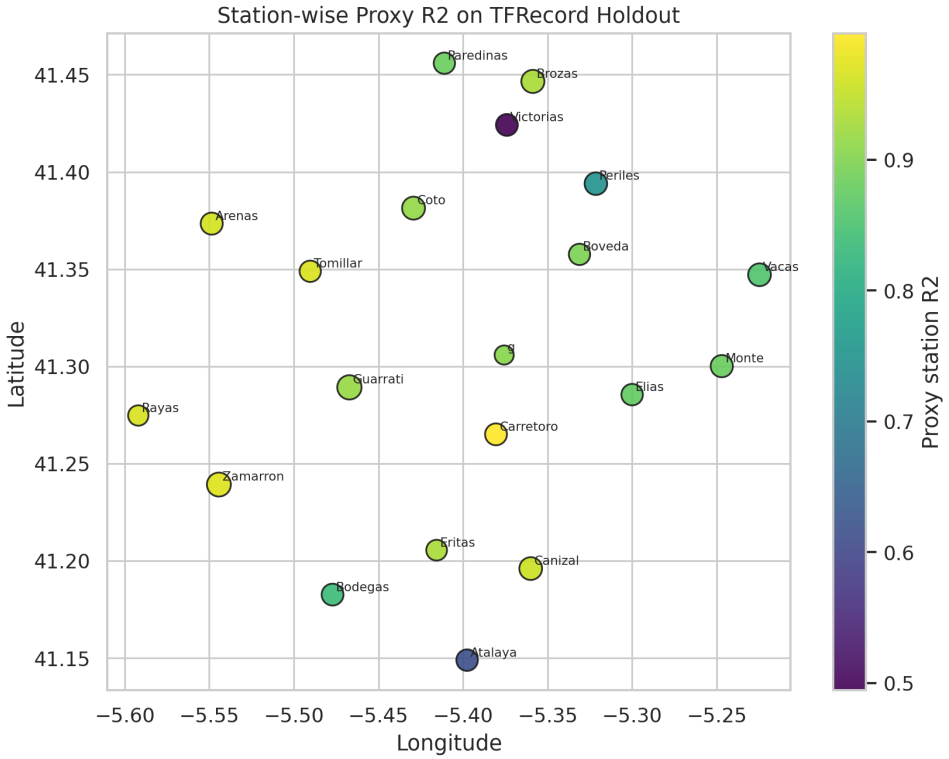


Figure 11.2: Station-wise proxy performance map over REMEDHUS coordinates (point color: per-station proxy R^2 on TFRecord holdout; marker size: test-sample count).

11.4 Comparative Summary

Table 11.1: Spain baseline vs final summary

Model	Split	R^2	Error Metric
Residual XGBoost baseline	Station	0.3619	RMSE = 0.0830
MultiModalSoilNet (selected)	Station	0.7016	NRMSE = 6.21%
MultiModalSoilNet (diagnostic)	Temporal	0.8656	NRMSE = 3.53%

Chapter 12

Limitations

12.1 Measurement and Representation Limits

Surface soil moisture estimates in this study face an inherent representativeness mismatch: in situ labels are point measurements, while predictor products are often area averages at roughly kilometer scale. In heterogeneous agricultural landscapes, that scale mismatch can create irreducible error even when both labels and predictors are high quality. So part of the residual error budget comes from observation geometry, not only model limitations.

12.2 Sampling Bias and Coverage Limits

Station placement is not spatially random, and may over-represent specific management regimes, soil classes, or microclimates. As a result, performance can degrade in out-of-distribution settings, such as sparse-station areas, regions with different crop calendars, or different hydro-climatic forcing regimes. This is especially relevant when extrapolating beyond the core monitoring footprint.

12.3 Temporal and Sensor Availability Constraints

Cloud contamination, revisit schedules, and product latency reduce effective daily signal availability and increase dependence on fallback compositing. Even with row-preservation safeguards, these mechanisms can smooth short-lived extremes or reduce signal sharpness during sparse periods. The resulting uncertainty is episodic rather than uniform, and it often varies by season.

12.4 Model Complexity Tradeoff

The multimodal deep architecture delivers strong predictive gains, but it also increases implementation complexity, computational demand, and operational monitoring needs. Compared with compact tabular baselines, this system needs stricter controls around data contracts, calibration governance, and retraining diagnostics. Deployment decisions therefore need to balance predictive lift against maintenance and infrastructure cost.

Chapter 13

Future Work

13.1 Further Downscaling from 1 km to 500 m

One practical next step is a second-stage framework that refines current 1 km outputs to 500 m. The existing model would provide a stable first-stage estimate, and a dedicated refinement model would learn sub-kilometer residual structure from higher-resolution auxiliary predictors where available. The objective is to keep large-scale consistency while adding field-level detail that is useful for management decisions.

13.2 Expansion to Global Generalization

The next strategic objective is to move from regional deployment toward globally generalizable performance. Reaching that point will require harmonized multi-region training data, climate-zone-aware validation protocols, and strict out-of-region tests that measure transfer honestly. Even at global scale, the residual-anchoring strategy should remain, with stronger adaptation mechanisms for diverse land-surface, vegetation, and hydro-climatic regimes.

13.3 Improvement of Unseen-Station Performance

Improving strict unseen-station performance remains a top priority. Future work will focus on reducing transfer error through stronger domain generalization, targeted calibration improvements, and controlled ablation-guided architecture refinement. Evaluation will remain centered on station-disjoint protocols so metric gains reflect real transfer capability, not station-specific memorization.

Chapter 14

Conclusion

This report documents a full end-to-end progression from a robust baseline to a strong station-generalization deep model for Spain. The final system exceeds the project target under strict unseen-station evaluation and shows that structured multimodal learning with residual anchoring can substantially improve surface soil-moisture prediction quality.

Beyond the final metrics, the key contribution is methodological discipline: quality-controlled labels, leakage-aware splits, explicit feature routing, robust optimization, and reproducibility-minded evaluation. Just as importantly, these choices make the pipeline easier to trust, extend, and adapt for future scientific and operational use.

Bibliography

[1] Vishal Batchu, Grey Nearing, and Varun Gulshan. A deep learning data fusion model using sentinel-1/2, soilgrids, smap, and gldas for soil moisture retrieval. *Journal of Hydrometeorology*, 24(10):1789 – 1823, 2023. doi: 10.1175/JHM-D-22-0118.1. URL https://journals.ametsoc.org/view/journals/hydr/24/10/JHM-D-22-0118.1.xml.

[2] Kuai Fang, Chaopeng Shen, Daniel Kifer, and Xiao Yang. Prolongation of smap to spatiotemporally seamless coverage of continental u.s. using a deep learning neural network. *Geophysical Research Letters*, 44(21):11,030–11,039, 2017. doi: https://doi.org/10.1002/2017GL075619. URL https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1002/2017GL075619.

[3] Dharmendra Pandey, Prashant Srivastava, Rucha Dave, Praveen Gupta, and Rashmi Sharma. Improvised sar time series algorithm for nisar high resolution soil moisture product over indian tropical region. pages 2111–2115, 08 2025. doi: 10.1109/IGARSS55030.2025.11313937.

[4] Zeyang Wei, Lifei Wei, Ting Wang, Qikai Lu, Shuang Tian, Fei Zhang, and Yanfei Zhong. A global long-term daily multilayer soil moisture dataset derived from machine learning. *Scientific Data*, 13(1):122, Dec 2025. ISSN 2052-4463. doi: 10.1038/s41597-025-06436-0. URL https://doi.org/10.1038/s41597-025-06436-0.

[5] Wei Xu, Zhaoxu Zhang, Zehao Long, and Qiming Qin. Downscaling smap soil moisture products with convolutional neural network. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 14:4051–4062, 2021. doi: 10.1109/JSTARS.2021.3069774.